CI/CD for Android — Zero to Interview

A baby-steps guide. No prior knowledge needed. Read in order.

Beginner-friendly | 3 example pipelines | Play Store publishing | AI-era tips

About this sheet: a friendly, walk-you-through-it guide to CI/CD for Android — written for someone who has heard the words but never set one up. Read top to bottom. Each section builds on the previous one. Original prose, real example YAML files, and an interview Q&A at the end.

1. What is CI/CD? — explain like I'm 5

Imagine you and 4 friends are building a house together. Every day, each person builds a different room. At the end of the week, you all try to put the rooms together — but the doors don't line up, the wiring doesn't match, the plumbing is on the wrong side. Hours wasted fixing collisions.

Now imagine instead: every time someone finishes a small piece, they bring it to a robot in the middle of the site. The robot:

1. Tries to fit the piece with everything else built so far
2. If it fits → keeps it
3. If it doesn't → buzzes loudly so the person can fix it immediately

That robot is **Continuous Integration (CI)**.

Now imagine that same robot, after every successful fit, also **moves the assembled house to a showroom** for the team to walk through (staging), and once a week to the actual **buyer's lot** (Play Store).

That extra moving step is **Continuous Delivery / Deployment (CD)**.

> **One-line definitions:**
>
> - **CI = Continuous Integration** — every code change is automatically built + tested + verified to fit with the rest of the codebase, on every push.
> - **CD = Continuous Delivery** — every successful CI build automatically gets *ready* to be deployed (you click a button to deploy).
> - **CD = Continuous Deployment** — every successful CI build automatically goes to production (no button, fully automatic).

Most teams use **CI + Continuous Delivery** (build + test automatic, deployment is a button push). Apps that ship multiple times a day (Netflix, Amazon) use full Continuous Deployment.

2. Why do we need CI/CD? — what would happen without it

Picture an Android team without CI/CD:

1. Developer writes code on their machine. Builds and tests it locally. Works for them.
2. Pushes code to git.
3. Two days later, someone else tries to build the app. **It fails on their machine** ("works on my machine!").
4. QA needs an APK to test. Developer manually runs `./gradlew assembleDebug`, finds the APK, drops it in Slack. Three times a day.
5. Release time. Developer runs `./gradlew bundleRelease`, manually signs it, opens Play Console, manually uploads, manually fills release notes, manually rolls out. **30 manual steps. Easy to skip one.**
6. Production crash because a unit test that everyone forgot to run was failing for 2 weeks.

Now with CI/CD:

1. Developer pushes code.
2. A robot (CI server) automatically: pulls the code, builds it, runs all tests, runs lint, checks code coverage, builds an APK + AAB.
3. QA gets the APK in Slack automatically.
4. If you push to `main`, it auto-uploads to Play Store internal track.
5. Bugs caught the moment they're introduced — not 2 weeks later.

WITHOUT CI/CD	WITH CI/CD
"Works on my machine"	Builds verified on a clean server
Manual APK building & sharing	Auto-uploaded to QA / testers
Tests forgotten / skipped	Tests run on every push
Manual Play Store releases	One git push → user has the update tomorrow
Bugs found in production	Bugs caught at PR time
Slow team velocity	Multiple deploys per day possible

3. The CI/CD pipeline — what actually happens (visualized)

A **pipeline** is a series of automated steps. Each step's output feeds the next. Here's a typical Android pipeline:

```
DEVELOPER PUSHES CODE TO GIT
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────┐
│  CI SERVER WAKES UP (triggered by push or PR)                    │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 1. CHECKOUT  │  Pull latest code from git
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 2. SETUP     │  Install Java 17, Android SDK, etc.
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 3. RESTORE   │  Restore Gradle cache from previous builds
                      │    CACHE     │  (saves ~5-10 minutes)
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 4. LINT      │  ./gradlew lint
                      │              │  Catches Android issues
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 5. UNIT      │  ./gradlew testDebugUnitTest
                      │    TESTS     │  Fast JVM tests
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 6. UI / E2E  │  ./gradlew connectedAndroidTest
                      │    TESTS     │  Run on emulator/device
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 7. BUILD APK │  ./gradlew bundleRelease
                      │    + AAB     │  Sign with release keystore
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 8. UPLOAD    │  Save APK as build artifact
                      │  ARTIFACTS   │  Available to download for 30+ days
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 9. DEPLOY    │  Upload AAB to Play Store via API
                      │  (CD)        │  Track: internal / alpha / beta / prod
                      └──────────────┘
                              │
                              ▼
                      ┌──────────────┐
                      │ 10. NOTIFY   │  Slack message:
```

```
                          │               │  "✅ Build #142 deployed to internal track"
                          └───────────────┘
```

Each box is a "step" or "job" in CI/CD vocabulary. If any step fails (red ❌), the pipeline stops there and notifies you.

4. Key vocabulary — the words you'll keep hearing

Pipeline
The full series of automated steps from "code pushed" to "users have the update."

Job / Stage / Step
One unit of work in the pipeline. Different tools call it slightly differently. Roughly: a step = one shell command; a job = a group of steps; a stage = a group of jobs (e.g., "Test stage" → "build job", "lint job", "test job").

Workflow
A complete pipeline configuration, usually defined in a YAML file (like `.github/workflows/android.yml`).

Trigger
What starts the pipeline. Common triggers: push to a branch, PR opened, scheduled (every night), manually clicked.

Runner / Agent / Worker
The actual machine that runs your pipeline. Could be a cloud Linux VM (free 2000 min/month on GitHub Actions), a Mac mini in your office, or a Bitrise mac. Android needs a runner with the SDK installed.

Artifact
A file produced by the pipeline that you want to keep — your APK, AAB, test report, mapping.txt, screenshots.

Cache
Files saved from one pipeline run, restored in the next. Gradle cache, dependencies cache, etc. — saves a lot of time.

Secret / Environment variable
Sensitive data the pipeline needs but you can't put in git: signing keystore password, Play Store API key, Slack webhook URL. Stored encrypted in the CI tool.

> **Track / Channel**
> Play Store concept. Internal → Closed alpha → Closed beta → Open beta → Production. CI/CD usually pushes to "internal" first; promote manually after smoke testing.

5. CI/CD tools / websites in the market

You mentioned Jenkins. Here's the full landscape in 2026.

TOOL	HOSTED / SELF?	BEST FOR	PRICING (2026 BALLPARK)
GitHub Actions	Hosted (GitHub)	Most popular for any project on GitHub. Tight integration with PRs.	Free for public repos; 2000 min/month free for private; pay after
GitLab CI	Hosted (GitLab) or Self-hosted	If your code is on GitLab. YAML similar to GHA.	Free tier exists; paid for big teams
Bitrise	Hosted	**Mobile-first.** Best for Android+iOS in one place. Visual workflow builder + YAML.	Free for hobby; paid for teams (~$30+/mo per concurrency)
Codemagic	Hosted	Mobile-first like Bitrise; popular for Flutter + native. 500 mins/mo free.	Free tier; paid after
CircleCI	Hosted or Self-hosted	Generic but mobile-friendly. Powerful caching.	Free tier; paid for more
Jenkins	**Self-hosted**	Traditional choice. Highly customizable. Many plugins. You run the server yourself.	Free (open source); you pay for the server
Azure Pipelines	Hosted (Microsoft)	Enterprises with Microsoft stack	Free for open source; paid for private
TeamCity	Self-hosted (JetBrains)	Enterprises wanting JetBrains ecosystem	Free Professional tier; paid Enterprise
Buddy	Hosted	Drag-and-drop pipeline builder; simple	Free tier; paid
App Center (Microsoft)	Hosted	Mobile-first; **shutting down March 2025** — avoid for new projects	n/a

If you're starting today, pick one of these:

FOR A HOBBY / PERSONAL PROJECT

- **GITHUB ACTIONS**
  if your code is on GitHub — free, easy YAML, tons of community examples
- 2000 free Linux minutes/month is enough for most personal projects

FOR A SMALL TEAM / STARTUP

- **BITRISE**
  if you want mobile-specific (Android + iOS) and a visual workflow builder
- **GITHUB ACTIONS**
  if you have backend + Android in same repo

FOR A BIG ENTERPRISE

- **JENKINS**
  on internal infrastructure (security/compliance)
- **GITLAB CI / AZURE PIPELINES**
  if already on GitLab/Azure
- **CIRCLECI**
  for cross-platform monorepos

FOR A FLUTTER / CROSS-PLATFORM APP

- **CODEMAGIC**
  — designed for Flutter; one workflow for Android + iOS
- **BITRISE**
  also good for Flutter

6. Do I need to learn a language? — what tools/skills are required

Good news: you mostly don't need to learn a new programming language. Here's what you actually need:

SKILL	HOW MUCH	WHY
YAML	2-3 hours of learning	Almost every modern CI/CD tool (GitHub Actions, Bitrise, GitLab CI, CircleCI) defines pipelines in YAML files. Just nested key-value pairs with indentation.
Bash / shell	Basic familiarity	Steps usually run shell commands: `./gradlew build`, `cp`, `curl`. Don't need to be an expert.
Gradle CLI	Basic commands	Know what `./gradlew assembleRelease`, `bundleRelease`, `testDebugUnitTest`, `lint` do (see Gradle cheat sheet)
Git basics	You already know	Branches, tags, commits, PRs
Jenkins (only if Jenkins): Groovy DSL	Read existing scripts	Jenkins pipelines (Jenkinsfile) are written in Groovy. You don't need to be fluent — just modify existing examples.
Fastlane (optional but recommended)	Just learn the lane syntax	Ruby tool that simplifies signing, screenshots, and Play Store uploads. Beautiful one-liners.
Docker (optional)	Basic concepts	For self-hosted runners, custom build environments. Not needed for hosted CI tools.

Tiny YAML primer (2 minutes)

```yaml
# Comment
name: "My Pipeline"           # key: value

on:                            # nested keys = indented (2 spaces standard)
  push:
    branches:
      - main                   # list items start with `-`
      - develop

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: "Checkout"       # dash = item; rest = key/values for that item
        uses: actions/checkout@v4

      - name: "Build"
        run: |                 # `|` means multi-line literal
          ./gradlew build
          echo "done"
```

That's it. You're now 80% there for reading any CI/CD YAML.

7. The AI era — using AI to write CI/CD scripts

Honest truth: **writing CI/CD scripts is one of the things AI is BEST at.** CI/CD configs are well-documented, formulaic, and have tons of training data. Use AI aggressively.

How to use AI effectively for CI/CD

1. **Start with a clear goal.** Don't ask "set up CI/CD". Ask: *"Write a GitHub Actions workflow that builds an Android app, runs unit tests, signs the release APK, and uploads to Play Store internal track on every push to main."*
2. **Give the AI your context.** Tell it: project structure, Gradle version, AGP version, Kotlin version, what variants exist, where the keystore lives, which Play Store track.
3. **Iterate on errors.** When the pipeline fails, paste the full error log into AI. Ask "what does this error mean and how do I fix it?". Most CI errors have one-line fixes.
4. **Always read what AI generated.** Don't trust blindly. Common AI mistakes:
   - Using outdated action versions (`actions/checkout@v2` when v4 is current)
   - Suggesting deprecated Gradle commands (`--no-daemon` when daemon is preferred)
   - Hardcoding secrets (it'll suggest paths; you must use the secrets store)
5. **Use Cursor / Copilot in your YAML files.** Type a comment `# step to upload APK to Play Store internal track` and let AI fill in the YAML. Verify, commit.
6. **Cross-check with the official docs.** AI can be confidently wrong. Always verify the action name and parameters at `github.com/actions` or the tool's docs.

Sample prompt template

```
I'm using GitHub Actions for an Android app. My setup:
- Kotlin 2.0.0, AGP 8.5.0, Gradle 8.7
- Variants: qaDebug, prodDebug, qaRelease, prodRelease
- Keystore stored as a base64 secret named RELEASE_KEYSTORE
- Play Store API JSON in secret PLAY_API_KEY
- Need to upload prodRelease AAB to internal track on every push to main

Write the .github/workflows/release.yml. Use latest action versions.
Add Gradle cache. Fail the build if unit tests fail.
```

AI will produce a working YAML in seconds. Test on a feature branch first.

> **The 80/20:** AI writes 80% of CI/CD config correctly on first try. The 20% you fix is usually: action version drift, secret name conventions, your project's specific gradle quirks. Total time saved per pipeline: hours.

8. The Android CI/CD pipeline — every phase explained

#	PHASE	COMMAND	WHAT IT DOES
1	**Checkout**	`git clone`	Pull the latest code from your branch
2	**Setup JDK**	Install Java 17 (or 21)	Gradle needs a JVM
3	**Setup Android SDK**	Install or use pre-installed SDK	AGP needs Android SDK + build tools
4	**Restore Gradle cache**	Auto via `actions/cache` or built-in	Skip re-downloading dependencies
5	**Decode keystore**	Base64-decode secret to file	Sign release builds (only when releasing)
6	**Lint**	`./gradlew lint`	Catch Android-specific issues (deprecated API, missing translations, etc.)
7	**Unit tests**	`./gradlew testDebugUnitTest`	Run JVM-only tests; fast (seconds)
8	**Compose snapshot tests**	`./gradlew verifyPaparazziDebug`	Catch visual regressions on JVM
9	**Instrumented tests**	`./gradlew connectedDebugAndroidTest`	Run on emulator/device. Slow. Often skipped on PR; run nightly.
10	**Build AAB**	`./gradlew bundleRelease`	Produce the App Bundle for Play Store
11	**Upload artifacts**	Tool-specific (e.g., `actions/upload-artifact`)	Save AAB, mapping.txt, test reports as downloadable artifacts
12	**Upload mapping.txt**	To Crashlytics + Play Console	Required for symbolicated crash traces
13	**Deploy to Play Store**	Gradle Play Publisher or Fastlane	Auto-upload AAB to internal/alpha/beta/prod
14	**Notify Slack**	Webhook POST	Tell the team the build succeeded / failed

Not every push needs every phase. Common pattern:

- **On every push to feature branch / PR**: lint + unit tests + build debug APK
- **On push to `main`**: above + upload artifacts + maybe deploy to internal track
- **On git tag (e.g., `v1.4.2`)**: full release — sign + AAB + upload mapping + deploy to production track
- **Nightly** (scheduled): full instrumented test suite + Macrobenchmark

9. Example 1 — GitHub Actions (the simplest setup)

Create `.github/workflows/android.yml` in your repo:

```yaml
name: Android CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: "Checkout code"
        uses: actions/checkout@v4

      - name: "Set up JDK 17"
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"

      - name: "Set up Gradle (with cache)"
        uses: gradle/actions/setup-gradle@v3

      - name: "Run lint"
        run: ./gradlew lintDebug

      - name: "Run unit tests"
        run: ./gradlew testDebugUnitTest

      - name: "Build debug APK"
        run: ./gradlew assembleDebug

      - name: "Upload APK"
        uses: actions/upload-artifact@v4
        with:
          name: app-debug
          path: app/build/outputs/apk/debug/*.apk
```

Push this file. Go to **your repo → Actions tab**. You'll see your build running. After ~3-5 minutes, you can download the APK from the run page.

That's it. You have CI now.

Adding Play Store deployment to the same file

```yaml
deploy:
  runs-on: ubuntu-latest
  needs: build                    # wait for build job to succeed
  if: github.ref == 'refs/heads/main'  # only on main branch
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with: { distribution: temurin, java-version: "17" }
    - uses: gradle/actions/setup-gradle@v3

    - name: "Decode keystore"
      env:
        KEYSTORE_BASE64: ${{ secrets.RELEASE_KEYSTORE }}
      run: |
        echo "$KEYSTORE_BASE64" | base64 --decode > release.keystore

    - name: "Build & sign AAB"
      env:
        KEYSTORE_PWD:  ${{ secrets.KEYSTORE_PWD }}
        KEY_ALIAS:     ${{ secrets.KEY_ALIAS }}
        KEY_PWD:       ${{ secrets.KEY_PWD }}
      run: ./gradlew bundleRelease

    - name: "Upload AAB to Play Store internal track"
      uses: r0adkll/upload-google-play@v1
      with:
        serviceAccountJsonPlainText: ${{ secrets.PLAY_API_KEY }}
        packageName: com.example.app
        releaseFiles: app/build/outputs/bundle/release/app-release.aab
        track: internal
```

Now every push to `main` auto-deploys to Play Store internal track. Beta testers see the update within minutes.

10. Example 2 — Bitrise (mobile-first hosted)

Bitrise has a visual workflow editor — you don't have to write YAML manually. But the resulting `bitrise.yml` looks like this:

```yaml
format_version: "11"
default_step_lib_source: https://github.com/bitrise-io/bitrise-steplib.git
project_type: android

workflows:
  primary:
    steps:
      - activate-ssh-key@4: {}
      - git-clone@8: {}
      - restore-gradle-cache@1: {}
      - install-missing-android-tools@3:
          inputs:
            - gradlew_path: $GRADLEW_PATH
      - android-lint@0:
          inputs:
            - variant: debug
      - android-unit-test@1:
          inputs:
            - variant: debug
      - android-build@1:
          inputs:
            - variant: release
            - build_type: aab
      - sign-apk@1:
          inputs:
            - android_app: $BITRISE_AAB_PATH
      - google-play-deploy@3:
          inputs:
            - service_account_json_key_path: $PLAY_KEY_PATH
            - package_name: com.example.app
            - track: internal
      - save-gradle-cache@1: {}
      - deploy-to-bitrise-io@2: {}
      - slack@4:
          inputs:
            - webhook_url: $SLACK_WEBHOOK_URL
            - channel: "#android-builds"
```

Each `step` is a versioned, pre-built bundle from Bitrise's "Step Library." You connect them; secrets live in the Bitrise dashboard. Much less YAML to write yourself than GitHub Actions.

11. Example 3 — Jenkins (the classic, self-hosted)

Jenkins pipelines live in a `Jenkinsfile` at the repo root. They're written in Groovy DSL (yes, the same Groovy you knew from old build.gradle).

```
pipeline {
    agent { label 'android' }   // runs on a Jenkins agent labeled "android"

    environment {
        JAVA_HOME = '/opt/java/openjdk-17'
        ANDROID_HOME = '/opt/android-sdk'
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }

        stage('Lint') {
            steps { sh './gradlew lintDebug' }
        }

        stage('Unit Tests') {
            steps { sh './gradlew testDebugUnitTest' }
            post {
                always { junit 'app/build/test-results/**/*.xml' }
            }
        }

        stage('Build AAB') {
            steps {
                withCredentials([
                    file(credentialsId: 'release-keystore', variable: 'KEYSTORE_PATH'),
                    string(credentialsId: 'keystore-pwd', variable: 'KEYSTORE_PWD')
                ]) {
                    sh './gradlew bundleRelease'
                }
            }
        }

        stage('Deploy to Play Store') {
            when { branch 'main' }
            steps {
                withCredentials([
                    file(credentialsId: 'play-api-key', variable: 'PLAY_KEY_PATH')
                ]) {
                    sh './gradlew publishReleaseBundle'      // uses Gradle Play Publisher
                }
            }
        }
    }

    post {
        success { slackSend channel: '#android-builds', message: "✅ Build #${env.BUILD_NUMBER} succeeded" }
        failure { slackSend channel: '#android-builds', message: "❌ Build #${env.BUILD_NUMBER} failed" }
```

```
    }
}
```

Differences from hosted tools:

- You install Jenkins yourself (a JAR file or Docker image). One-time setup ~1 day if new.
- You provision agents (machines that run jobs) — could be a Mac mini for iOS, Linux VMs for Android.
- Secrets stored via "Credentials" plugin. `withCredentials` injects them as env vars.
- You patch + maintain Jenkins yourself. Sometimes painful. Lots of plugins.

12. Signing builds in CI — the keystore + secrets dance

Release APKs/AABs must be signed with your release keystore. The CI server doesn't have your keystore on disk. So:

1. **Convert keystore to base64:**

   ```
   base64 -i release.keystore -o release.keystore.b64
   ```

2. **Add to CI secrets:**
   - GitHub Actions: Settings → Secrets and variables → Actions → New repository secret. Name: `RELEASE_KEYSTORE`. Value: paste the base64 string.
   - Bitrise: App settings → Code signing tab. Bitrise has a built-in keystore manager.
   - Jenkins: Manage Jenkins → Credentials → add as "Secret file" type.

3. **Add the passwords too:** `KEYSTORE_PWD`, `KEY_ALIAS`, `KEY_PWD`.
4. **In the pipeline, decode and use:**

   ```yaml
   - name: Decode keystore
     env:
       KEYSTORE_BASE64: ${{ secrets.RELEASE_KEYSTORE }}
     run: echo "$KEYSTORE_BASE64" | base64 --decode > ${{ github.workspace }}/release.keystore

   - name: Build signed AAB
     env:
       RELEASE_STORE_FILE: ${{ github.workspace }}/release.keystore
       RELEASE_STORE_PASSWORD: ${{ secrets.KEYSTORE_PWD }}
       RELEASE_KEY_ALIAS:     ${{ secrets.KEY_ALIAS }}
       RELEASE_KEY_PASSWORD:  ${{ secrets.KEY_PWD }}
     run: ./gradlew bundleRelease
   ```

5. **In `app/build.gradle.kts`, read those env vars:**

```
android {
    signingConfigs {
        create("release") {
            storeFile = file(System.getenv("RELEASE_STORE_FILE") ?: "missing.keystore")
            storePassword = System.getenv("RELEASE_STORE_PASSWORD")
            keyAlias = System.getenv("RELEASE_KEY_ALIAS")
            keyPassword = System.getenv("RELEASE_KEY_PASSWORD")
        }
    }
    buildTypes {
        getByName("release") {
            signingConfig = signingConfigs.getByName("release")
        }
    }
}
```

> **Don't:** commit the keystore to git. Ever. Even in a private repo. Anyone with read access can extract it. Always go through encrypted CI secrets.

13. Publishing to Play Store via script

Two popular ways:

Option A — Gradle Play Publisher (the simplest for Gradle-based builds)

A Gradle plugin that adds tasks like `publishReleaseBundle` straight to your project.

```
// app/build.gradle.kts
plugins {
    id("com.github.triplet.play") version "3.10.1"
}

play {
    serviceAccountCredentials.set(file("play-api-key.json"))
    track.set("internal")         // or "alpha", "beta", "production"
    defaultToAppBundles.set(true)
    releaseStatus.set(ReleaseStatus.COMPLETED)
}
```

Then in CI:

```yaml
- name: Publish to Play Store
  env:
    PLAY_API_KEY: ${{ secrets.PLAY_API_KEY }}
  run: |
    echo "$PLAY_API_KEY" > play-api-key.json
    ./gradlew publishReleaseBundle
```

Option B — Fastlane (Ruby tool, more features)

Industry-standard for mobile CI. Cross-language, supports both Android + iOS.

```ruby
# fastlane/Fastfile
default_platform(:android)

platform :android do
  desc "Deploy to Play Store internal track"
  lane :internal do
    gradle(task: "bundleRelease")
    upload_to_play_store(
      track: "internal",
      aab: "app/build/outputs/bundle/release/app-release.aab",
      json_key: "play-api-key.json"
    )
  end
end
```

Then in CI: `fastlane internal`. Done.

Getting the Play Store service account key

1. Go to Play Console → Setup → API access
2. Click "Create new service account" → Google Cloud Console opens
3. Create a service account, give it a JSON key, download
4. Back in Play Console, grant that service account "Release manager" or "Admin" role
5. The JSON file goes into your CI secrets as `PLAY_API_KEY`

14. Fastlane — what it is and why it's loved

Fastlane is a Ruby tool that simplifies common mobile-release tasks into one-liners called **lanes**. It's not a CI/CD platform itself — it runs INSIDE your CI/CD. But it makes the steps so much simpler that most teams use it.

ACTION	WITHOUT FASTLANE	WITH FASTLANE
Build APK	Multi-line gradle invocation	`gradle(task: "assemble", build_type: "Release")`
Sign APK	Manual jarsigner / apksigner	Auto via `gradle` action
Upload to Play Store	50 lines of Google API code	`upload_to_play_store(track: "internal")`
Take screenshots for store	Manual via 6 devices	`screengrab` automates it
Deploy to Firebase App Distribution	Custom script	`firebase_app_distribution(...)`

Common Fastlane lanes for Android

```
# fastlane/Fastfile
default_platform(:android)

platform :android do

  # Lane 1: distribute beta build to Firebase
  lane :beta do
    gradle(task: "assemble", build_type: "Release")
    firebase_app_distribution(
      app: ENV["FIREBASE_APP_ID"],
      groups: "qa, internal-testers",
      release_notes: "Auto build from CI"
    )
  end

  # Lane 2: deploy to Play internal
  lane :internal do
    gradle(task: "bundle", build_type: "Release")
    upload_to_play_store(track: "internal")
  end

  # Lane 3: promote internal → production
  lane :promote do
    upload_to_play_store(
      track: "internal",
      track_promote_to: "production"
    )
  end

end
```

Then trigger from CI: `fastlane internal`, `fastlane beta`, etc.

15. Branching strategies — when each pipeline runs

Your CI/CD pipeline is triggered by git events. Different branching strategies = different trigger setups.

STRATEGY	BRANCHES	HOW CI/CD PLUGS IN
Trunk-based	Just `main` + short-lived feature branches	Every push to main → build + deploy to internal track. Tags → deploy to prod.
GitFlow	`main`, `develop`, `feature/*`, `release/*`, `hotfix/*`	PR → develop = run tests. Push to develop = deploy to QA. Push to main = deploy to prod.
GitHub Flow	`main` + feature branches	PR → main = run all tests. Merge to main = deploy.
Release branches	`main` + `release/v1.4` per version	Each release branch has its own pipeline; main pipeline is for development

For a small team in 2026, **trunk-based + feature branches + GitHub Flow** is the simplest and most common. CI/CD config:

- PR opened → run lint + unit tests + build debug APK
- Merged to `main` → above + build signed AAB + upload to Play Store internal
- Git tag `v1.4.2` created → above + promote to Play Store production

16. Environments — dev / QA / staging / production

Most apps have multiple environments. Each has different backend URLs, different API keys, different Firebase projects.

ENVIRONMENT	WHO USES IT	WHERE IN PIPELINE
Dev	Developers	Local builds, sometimes ad-hoc CI builds
QA / Staging	QA team, internal testers	Auto-deployed on merge to `develop` or `main`; via Firebase App Distribution / Play Internal track
Beta	External beta testers	Manually promoted from QA after smoke testing; via Play Closed/Open Beta track
Production	Real users	Manually triggered (or on tag) for important releases; staged rollout (10% → 50% → 100%)

How environments map to your build: usually **product flavors** (see Gradle cheat sheet §11). Each flavor has different `buildConfigField("BASE_URL", ...)`, different applicationId suffix, different Firebase google-services.json.

17. Secrets management — what counts as a secret + where to put it

SECRET TYPE	EXAMPLES	WHERE TO STORE
Signing material	release.keystore, KEY_PWD, KEYSTORE_PWD, KEY_ALIAS	CI tool's secrets store
Play Store	service account JSON	CI tool's secrets store
Backend keys	Sentry DSN, Crashlytics API token, Maps API key	Per-flavor BuildConfig field, value from CI secret
Notifications	Slack webhook URL, Discord webhook	CI tool's secrets store
App secrets	OAuth client secret, Stripe secret key	**Don't put in app at all** — back-end only

Where to store CI secrets

- **GitHub Actions**: Settings → Secrets and variables → Actions
- **Bitrise**: Code signing tab + Workflow editor → Env Vars (toggle "Make this Env Var protected")
- **Jenkins**: Manage Jenkins → Credentials
- **GitLab**: Settings → CI/CD → Variables (mark "Protected" + "Masked")
- **Larger teams**: HashiCorp Vault, AWS Secrets Manager — pulled into pipeline at runtime

> **Common mistakes:** printing secret values in logs (CI tools usually mask them but not always); commiting `local.properties` with API keys; checking google-services.json into a public repo (it contains your project ID + sender ID — not the worst, but considered bad).

18. Tests in CI — what runs where

TEST TYPE	WHERE IT RUNS IN CI	WHY
Unit tests (JVM)	Every PR / push (fast)	Catches logic bugs in seconds
Compose tests (with Robolectric)	Every PR / push (also fast)	Catches UI logic without emulator
Screenshot tests (Paparazzi/Roborazzi)	Every PR (medium)	Catches design regressions
Instrumented tests (emulator/device)	Nightly OR on merge to main	Slow and flaky; not on every PR
Macrobenchmark	Nightly on physical device	Perf regressions; needs real device
E2E (Maestro / UI Automator)	Pre-release	Critical user flows; slow

Reports go to: JUnit XML (parsed by all CI tools to show test results), HTML reports for humans, optionally Sonar/Codecov for coverage.

Showing test results in PR

```yaml
- name: "Publish test results"
  if: always()          # run even if tests failed
  uses: EnricoMi/publish-unit-test-result-action@v2
  with:
    files: "**/build/test-results/test*UnitTest/*.xml"
```

This adds a comment on the PR with test counts and failed test names. Excellent for code review.

19. Docker for self-hosted runners (only if Jenkins / GitLab self-hosted)

If you're using a self-hosted runner, you'll often run jobs inside a Docker container. The container has Java + Android SDK + Gradle pre-installed. Public images:

- `cimg/android:2024.01-node` — CircleCI's image, well-maintained
- `reactivecircus/android-emulator-runner` — Has emulators preinstalled (for instrumented tests)
- You can also build your own based on `openjdk:17` + sdkmanager install

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    container: cimg/android:2024.01      # Run inside this Docker image
    steps:
      - uses: actions/checkout@v4
      - run: ./gradlew build
```

Hosted runners (GitHub, Bitrise) usually come with everything pre-installed — you don't need Docker for the simple cases.

20. Best practices & common mistakes

DO

- Cache Gradle (`actions/setup-gradle`, `save-gradle-cache`)
- Run unit tests on every PR; instrumented tests nightly
- Fail fast — if lint fails, don't bother building
- Upload artifacts (APK, mapping.txt, test reports) so QA can grab them
- Notify Slack on success and failure
- Use the same Gradle/AGP/Kotlin version locally and on CI
- Use staged rollouts (10% → 50% → 100%) for production
- Tag releases in git so you can find them later
- Set up branch protection — main can't merge if CI fails
- Use AI to generate first draft of YAML; then verify
- Time-bound jobs (e.g., timeout-minutes: 30) so a hang doesn't burn your CI budget

DON'T

- Commit the keystore to git
- Print secrets in logs (mask them in tool config)
- Skip caching (your CI bill will hate you)
- Run instrumented tests on every PR (too slow + flaky)
- Hardcode Play Store release notes in the pipeline (use a file)
- Use `--no-daemon` in Gradle CI (was needed pre-Gradle 7; not anymore)
- Forget to upload mapping.txt — crash traces become unreadable
- Auto-promote to production without a human check
- Use the `latest` tag for action versions (versions can change unexpectedly)
- Run heavy work in `after_failure` hooks (slows recovery loop)
- Mix env-specific config in main code (use product flavors)

21. Interview Q&A — what to expect

Q: "What is CI/CD and why does it matter?"

Continuous Integration = every code push automatically builds + tests. Continuous Delivery = every successful build is ready to deploy. Continuous Deployment = every successful build automatically goes to production. Why: catches bugs the moment they're introduced (instead of weeks later), eliminates "works on my machine," automates releases, lets the team ship multiple times a day with confidence.

Q: "What CI/CD tools have you used?"

Be honest. Talk through what you actually used, what you set up, what problems you debugged. Even "I set up GitHub Actions for a side project" is valid. If you've worked with multiple, compare them: Bitrise's mobile-first focus and visual editor vs GitHub Actions' tight git integration vs Jenkins' flexibility but operational burden.

Q: "Walk me through the CI/CD pipeline for an Android app."

Go phase by phase: trigger (push to main) → checkout → setup JDK + Android SDK → restore Gradle cache → lint → unit tests → build AAB → sign with keystore from secrets → upload to Play Store internal via Gradle Play Publisher / fastlane → notify Slack. Bonus: mention that instrumented tests run nightly, screenshot tests on every PR, prod releases happen on git tag with staged rollout.

Q: "How do you store the release keystore in CI?"

Convert keystore to base64; store as encrypted secret in CI tool (GitHub Secrets, Bitrise Code Signing, Jenkins Credentials); decode at job runtime to a temp file; pass passwords as separate secret env vars; build.gradle.kts reads them via `System.getenv`. Never commit the keystore to git.

Q: "How do you deploy to Play Store from CI?"

Two ways. (1) **Gradle Play Publisher plugin** — adds `publishReleaseBundle` task; needs a service-account JSON in CI secrets. (2) **Fastlane** with `upload_to_play_store` — same idea, more features. Either way: create service account in Google Cloud, grant Play Console "Release manager" role, store the JSON as a CI secret.

Q: "How do you handle multiple environments (dev/QA/prod) in CI?"

Map environments to product flavors. Each flavor has its own backend URL, applicationId suffix, google-services.json. Pipeline picks the flavor based on the trigger: feature-branch CI builds qaDebug; main pushes deploy prodRelease to Play Internal; tags promote to production. Secrets per environment can use different secret names (PROD_KEYSTORE vs QA_KEYSTORE).

Q: "What are the differences between Continuous Delivery and Continuous Deployment?"

Continuous Delivery: every successful build is *ready* to be deployed; a human clicks "deploy." Continuous Deployment: every successful build automatically goes to production. Most apps use Delivery (gives a chance for last-mile QA); apps with strong test coverage and feature flags use Deployment (Netflix, Amazon).

Q: "How do you make CI builds faster?"

Cache Gradle (build cache, dependencies, wrapper); use KSP over kapt; parallel jobs; fail-fast (lint before tests, tests before build); skip instrumented tests on PRs; use larger CI runners for big projects; switch to remote build cache (Gradle Enterprise). Target: PR feedback under 10 minutes for a typical Android project.

Q: "What is fastlane and why use it?"

Ruby tool that bundles common mobile-release tasks (build, sign, upload to store, take screenshots, distribute beta) into one-liner "lanes." Saves tons of boilerplate. Cross-CI (works in any tool) and cross-platform (Android + iOS in one config). Industry standard for mobile.

Q: "How do you handle secrets in CI?"

Tool-specific encrypted secret store (GitHub Secrets, Bitrise Env Vars marked Protected, Jenkins Credentials, GitLab Variables marked Masked + Protected). Inject as env vars at runtime. Never log them; the CI tool masks them in logs but be careful with your own echo statements. Larger orgs use HashiCorp Vault or AWS Secrets Manager.

Q: "How do you debug a failing CI build?"

Read the logs first — most errors say exactly what's wrong. Common patterns: dependency version conflict (Gradle prints the conflict), missing secret (env var empty), wrong Java/Gradle version, sdkmanager license not accepted, OOM in the Gradle daemon (bump JVM args). For pesky issues: SSH into the runner if your tool supports it (GitHub Actions has `tmate` action), or reproduce locally with the same JDK/AGP/Gradle version.

Q: "What's a staged rollout and why use it?"

Play Store feature: release to a percentage of users first (e.g., 10%), wait, then expand to 50%, then 100%. If a critical bug shows up, halt the rollout — only those 10% are affected. CI/CD can be configured to start at a small percentage; humans expand based on Crashlytics/Vitals.

Q: "How would you set up CI/CD for a brand-new Android project?"

Start small: GitHub Actions YAML with checkout + JDK + Gradle + lint + unit tests + build debug APK. Push it, see it work. Then layer on: signing with keystore secret, build release AAB, upload to Play Store internal track via Gradle Play Publisher. Then add: nightly instrumented tests, screenshot tests, Slack notifications. Then: branch protection requiring CI to pass before merge. Build incrementally; don't try to do everything day one.

Q: "What's the role of artifacts in CI/CD?"

Artifacts are files the pipeline produces that you want to keep — APK, AAB, mapping.txt, test reports, screenshots. Most CI tools store artifacts for 30-90 days, downloadable from the run page. Key for: QA grabbing the latest APK; debugging failed releases; retroactive symbolication of crashes (mapping.txt). Always upload mapping.txt and at least the AAB.

22. One-page recall card

The mental model

CI = every code push is automatically built + tested. CD = every successful build is automatically ready (Delivery) or goes (Deployment) to users. Pipeline = sequence of automated steps in a YAML file. Trigger → Checkout → Setup → Cache → Build → Test → Sign → Upload → Notify.

Tools by use case

- Personal/hobby: GitHub Actions
- Mobile-first team: Bitrise or Codemagic
- Cross-platform monorepo: CircleCI
- Enterprise self-hosted: Jenkins or GitLab CI
- Microsoft stack: Azure Pipelines

Skills you need

- YAML (2-3 hours)
- Bash basics
- Gradle CLI (assembleRelease, bundleRelease)
- Optional: Fastlane lanes, Jenkins Groovy, Docker

Android-specific must-knows

- Sign with keystore from base64 secret
- Use product flavors for environments
- Upload AAB (not APK) to Play Store
- Always upload mapping.txt

- Use Gradle Play Publisher or Fastlane upload_to_play_store
- Track: internal → alpha → beta → production
- Staged rollout for prod (10% → 50% → 100%)

Performance tricks

- Cache Gradle
- Skip instrumented tests on PR (run nightly)
- Fail-fast (lint → tests → build)
- KSP over kapt
- Larger runners for big projects
- Remote build cache (Gradle Enterprise) at scale

AI era

- AI writes 80% of YAML correctly first try
- Iterate on errors: paste log → ask for fix
- Use Cursor/Copilot in YAML files
- Always read what AI generated; verify action versions

Avoid

- Keystore in git
- Secrets in logs
- No caching
- Auto-promote to prod without human
- Floating action versions (use pinned)
- Mix env config in main code (use flavors)

Read top to bottom for understanding · skim §20 (best practices), §21 (Q&A), §22 (recall) before any CI/CD interview. Pair with the Gradle Deep Dive cheat sheet in this repo.